Akhand Akhand

Akhand

Quick Submit

Quick Submit

Indian Institute of Technology Patna

Document Details

Submission ID

trn:oid:::1:3566908786

Submission Date

May 12, 2026, 12:01 AM GMT+5:30

Download Date

May 12, 2026, 12:06 AM GMT+5:30

File Name

Wireless_Assignment.pdf

File Size

1.6 MB

8 Pages

5,768 Words

31,019 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

turnitin Page 3 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

Combating Data Heterogeneity in Federated Learning: An Empirical Analysis of Shared Representations via FedRep

Akhand Singh, Anchal Dubey, Aditya Chauhan, Saksham Singh, Mayur Borse, Aditya Kumar Yadav

***Abstract*—Federated Learning (FL) lets a group of decentralized clients train a model together under the orchestration of a central server while keeping their raw data on-device. The catch is that performance often falls apart when the local data distributions across clients look very different from each other, a setting known as non-IID. This is the norm rather than the exception in real-world deployments. Canonical algorithms such as Federated Averaging (FedAvg) try to learn a single shared model for everyone, and that model ends up suffering from objective inconsistency and client drift when the underlying tasks pull in different directions. In this paper, we reproduce and empirically study the Federated Representation Learning (FedRep) algorithm, which splits the model into a shared low-dimensional global representation and a small client-specific classification head. We implement FedRep on top of the Flower (`flwr`) framework using PyTorch, and we benchmark it against a FedAvg baseline across a wide range of statistical heterogeneity controlled by a Dirichlet distribution with concentration parameter $\alpha \in \{0.01, 0.1, 0.5, 1.0, \text{IID}\}$. On the Fashion-MNIST dataset with 10 clients, our results show a striking inversion of performance. Under extreme heterogeneity (α=0.01), FedRep reaches a personalized accuracy of 98.15% while FedAvg manages only 62.50%, but FedAvg pulls ahead under fully IID conditions (91.13% vs. FedRep's 75.97%). On top of that, because only the representation layers travel across the network, FedRep cuts upload communication cost by over 99%. We discuss what this personalization trade-off means for federated system design and sketch directions for future work.**

***Index Terms*—Federated Learning, Personalization, Data Heterogeneity, Representation Learning, Non-IID, Flower Framework, Dirichlet Partitioning.**

I. Introduction

FEDERATED learning (FL) has become a leading paradigm for training machine learning models on data that lives across millions of edge devices, hospitals, or organizational silos [2]. The framework lets clients collaboratively learn a model by exchanging only model updates with a central server, never the underlying private data. Coordinated by the server, the canonical Federated Averaging (FedAvg) algorithm proceeds in rounds: a subset of clients downloads the current global model, performs several epochs of local stochastic gradient descent (SGD), and uploads its updated parameters; the server then computes a weighted average to form the next global iterate. This template has shown notable success in domains such as next-word prediction on mobile keyboards, healthcare imaging, and ad-recommendation, where data privacy and bandwidth constraints rule out centralized training.

Despite all this progress, one of the most persistent and consequential challenges in FL is the issue of *data heterogeneity*. In real deployments, the local data distributions $\mathcal{D}_i$ across clients are usually non-identically distributed (non-IID). When that happens, the local loss landscapes pull in different directions: each client's empirical risk minimizer drags the global iterate toward its own optimum. The result is *client drift*, where local updates partially cancel out during averaging, leading to slow, oscillatory, or stalled convergence. Even more fundamentally, even at the true global optimum a single shared model can perform poorly for any individual client, because no single set of parameters can simultaneously minimize all of the heterogeneous local risks. Several remedies have been proposed. FedProx [3] adds a proximal regularizer, while SCAFFOLD [4] uses control variates to correct for drift. But these methods still try to learn a single global model, and they inherit the same fundamental ceiling whenever clients genuinely disagree about what the right model is.

A more recent and conceptually different line of work treats FL not as the search for a universal model but as a collection of related learning tasks that share structure. Empirical evidence from centralized deep learning suggests that data of similar modality, such as images or text, admits a common low-dimensional feature representation, even when the downstream label distributions look very different. This intuition motivates the Federated Representation Learning (FedRep) algorithm of Collins et al. [1], which decomposes each client's model q_i as the composition $q_i = h_i \circ \phi$. Here, $\phi : \mathbb{R}^d \to \mathbb{R}^k$ is a global representation shared across clients and $h_i : \mathbb{R}^k \to \mathcal{Y}$ is a small client-specific head. By isolating the heterogeneity in the heads, which are kept strictly local and never transmitted, FedRep lets the shared representation be optimized cleanly across all clients, while each client tailors its head to its own label distribution.

In this paper, we carry out a careful empirical study of FedRep relative to the FedAvg baseline under a controlled gradient of statistical heterogeneity. Our specific contributions are:

1) We reproduce the FedRep algorithm within the Flower (`flwr`) framework, faithfully implementing the

A. P. N. Singh and A. Dubey are with the Department of Computer Science and Engineering, Indian Institute of Technology (IIT) Patna, Bihar, India. (e-mail: akhand_2311ai30@iitp.ac.in; anchal_2311ai01@iitp.ac.in).

Term paper prepared for the Federated Learning course (Jan to April 2026).

turnitin Page 3 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

turnitin Page 4 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

alternating-minimization client procedure (freeze the representation and train the head, then freeze the head and train the representation) on top of a convolutional backbone for Fashion-MNIST classification.

2) We benchmark FedRep against FedAvg under five regimes of heterogeneity induced by a Dirichlet distribution with concentration parameter $\alpha \in \{0.01, 0.1, 0.5, 1.0, \text{IID}\}$, holding all other experimental knobs fixed.
3) We empirically demonstrate a clear *personalization trade-off*. FedRep dominates by a wide margin under extreme heterogeneity, the two methods cross over near moderate heterogeneity, and FedAvg outperforms when data is essentially IID.
4) We quantify the communication-efficiency advantage of FedRep, showing a greater than 99% reduction in upload bandwidth relative to FedAvg over a 50-round simulation, without compromising accuracy in non-IID regimes (in fact substantially improving it).

The rest of this paper is organized as follows. Section II surveys related work in standard, robust, and personalized FL. Section III formalizes the heterogeneous FL objective and motivates the shared-representation formulation. Section IV describes the FedRep algorithm in detail and our Flower-based implementation. Section V presents the experimental protocol, and Section VI reports and discusses results. Section VII discusses limitations and future work, and Section VIII concludes.

II. Related Work

A. Standard Federated Learning under Heterogeneity

The standard formulation of FL aims to learn a single shared model $q = q_1 = \cdots = q_n$ that minimizes the weighted average of local empirical risks. FedAvg [2] approximates this objective by periodically averaging local SGD updates. Its communication efficiency comes from allowing multiple local epochs between communication rounds. But those same local epochs cause objective inconsistency under heterogeneity: each client's iterate drifts toward its own minimizer, and the server-side average can end up far from the global optimum.

To mitigate client drift, FedProx [3] augments the local objective with a quadratic penalty $\frac{\mu}{2}\|w - w^t\|^2$ that anchors local iterates to the current global model. SCAFFOLD [4] introduces server-side and client-side control variates that estimate and correct the drift direction at each step. FedNova [5] normalizes the local updates to remove bias arising from differing numbers of local steps, and FedDyn dynamically regularizes the local objectives to align stationary points with the global objective. All of these methods retain a single global model, and so they inherit a fundamental ceiling. When the optimal models for different clients genuinely differ, no shared parameter vector can perform well for every client simultaneously.

B. Personalized Federated Learning

A second body of work explicitly embraces personalization. Local fine-tuning starts from the global model and adapts it to each client's data, often with a few epochs of SGD on the local set. Meta-learning approaches such as Per-FedAvg [6] learn an initialization that is amenable to rapid local adaptation. Mixture-based approaches [7], [8] maintain both local and global models and interpolate between them, while multi-task formulations [9] learn task-specific models with a regularizer that encodes inter-client similarity. Ditto [10] couples a global model with personalized models constrained to stay close to it, and achieves a favorable robustness-accuracy trade-off. While these methods are effective, they all still solve a full-dimensional local problem at each client, so the sample-complexity benefit of cooperation among clients remains limited.

C. Representation Learning in Federated Settings

The idea of factorizing a model into a shared representation and task-specific heads is classical in multi-task learning. In federated settings, FedPer [11] was an early proposal that locally personalized the final layers of the network. However, FedPer updates the representation and the head jointly with the same number of local steps, and the original work provides no theoretical justification for that choice. LG-FedAvg [12] flips the decomposition by learning local representations and a global head, which is appropriate for some settings but less so when heterogeneity is concentrated in the labels.

FedRep [1], which we study in this paper, takes the complementary view: a single global representation ϕ shared by all clients, and many client-specific heads h_i. The key insight is that on each local round, FedRep first freezes ϕ and performs τ_h steps to solve (approximately) the inner minimization over h_i given the current ϕ. Only then does it unfreeze ϕ and take τ_ϕ steps to update the representation. The authors prove in a linear setting that this alternating minimization-descent procedure achieves near-optimal sample complexity per client, and they demonstrate empirically that FedRep outperforms a broad suite of personalized FL baselines on CIFAR-10, CIFAR-100, FEMNIST, and Sent140. Our work complements theirs by systematically charting the FedRep vs. FedAvg trade-off across a Dirichlet-controlled spectrum of heterogeneity.

D. Statistical Heterogeneity via Dirichlet Partitioning

For benchmarking, the community has converged on the Dirichlet partitioning scheme introduced by Hsu et al. [13]. For each class c, one samples a vector of proportions $\mathbf{p}_c \sim \mathrm{Dir}(\alpha)$ over clients. Small α produces highly skewed allocations, where a client may see only one or two classes, while large α approaches an even split. This parameterization gives us a continuous knob to study the effect of heterogeneity, and it is the protocol we adopt here.

III. Problem Formulation

A. Federated Learning Objective

We consider a federation of n clients, indexed by $i \in [n] := \{1, \ldots, n\}$. Each client i has a local data distribution $\mathcal{D}_i$ over input-label pairs $(x, y) \in \mathbb{R}^d \times \mathcal{Y}$, and has access to a finite

turnitin Page 4 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

turnitin Page 5 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

sample of size M_i drawn from $\mathcal{D}_i$. Let $q_i : \mathbb{R}^d \to \mathcal{Y}$ denote client i's learning model, and define the population risk

$$f_i(q_i) := \mathbb{E}_{(x,y)\sim\mathcal{D}_i}\big[\,\ell(q_i(x), y)\,\big], \tag{1}$$

where $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$ is a loss function. The generic FL problem is

$$\min_{(q_1,\ldots,q_n)\in\mathcal{Q}^n} \frac{1}{n}\sum_{i=1}^{n} f_i(q_i), \tag{2}$$

where $\mathcal{Q}$ is the space of feasible models. Standard FL approaches specialize (2) by enforcing the constraint $q_1 = \cdots = q_n = q$, reducing the problem to the search for a single shared model. This specialization is appropriate when the $\mathcal{D}_i$ are similar but is fundamentally limited under heterogeneity, as we now discuss.

B. The Shared-Representation Objective

We adopt the formulation of [1]. Suppose the model q_i decomposes as the composition of a global representation $\phi : \mathbb{R}^d \to \mathbb{R}^k$ (with $k \ll d$) and a client-specific head $h_i : \mathbb{R}^k \to \mathcal{Y}$, that is,

$$q_i(x) = (h_i \circ \phi)(x). \tag{3}$$

Because k is small, each client's head h_i has few parameters and can be learned from few local samples once ϕ is fixed. The corresponding federated objective is

$$\min_{\phi\in\Phi} \frac{1}{n}\sum_{i=1}^{n} \min_{h_i\in\mathcal{H}} f_i(h_i \circ \phi), \tag{4}$$

where Φ and $\mathcal{H}$ are the spaces of feasible representations and heads, respectively. The inner minimization is performed locally by each client; the outer minimization is solved collaboratively across the federation. The important point is that only ϕ ever needs to leave the client.

C. Why a Shared Representation Helps under Heterogeneity

Consider the linear specialization of (4) studied in [1]. Let $\phi(x) = B^\top x$ with $B \in \mathbb{R}^{d\times k}$ and $h_i(z) = w_i^\top z$ with $w_i \in \mathbb{R}^k$, and assume each $\mathcal{D}_i$ is induced by $y = w_i^{*\top} B^{*\top} x$ for some ground-truth $(B^*, \{w_i^*\})$. Under standard FL, the optimal shared solution (B, w) collapses the diversity of the w_i^* into their average, leaving a residual error that grows with heterogeneity. In contrast, the shared-representation objective (4) preserves a unique w_i per client and, provided $\{w_i^*\}$ span $\mathbb{R}^k$, can recover the column space of B^* exactly. The theoretical sample complexity per client scales as $\Theta(d/n + \log n)$, compared to $\Theta(d)$ for local-only learning. This shows that federation provides a strict benefit whenever $1 \ll n \ll e^{\Theta(d)}$.

IV. The FedRep Algorithm and Implementation

A. Algorithm Description

FedRep solves (4) by an alternating procedure that distributes computation across the federation. The pseudocode is given in Algorithm 1.

On each round, the server samples a fraction $r \in (0, 1]$ of clients and broadcasts the current representation. Each selected client first executes τ_h local gradient updates on its head, holding the representation fixed (lines 7 to 9). The intuition is that solving the inner minimization in (4) as well as possible before updating the representation guarantees that the representation-update gradient is computed at a near-optimal head, which yields a cleaner signal. Once the head is updated, the client executes τ_ϕ local steps on the representation, with the head frozen at its newly trained value (lines 11 to 13). Finally, the client uploads only the representation; the head h_i never leaves the device. The server averages the received representations to form the next iterate.

Algorithm 1 FedRep

Require: Participation rate r, step size α, head epochs τ_h, representation epochs τ_ϕ, communication rounds T.

1: Server initializes representation ϕ^0; each client i initializes head h_i^0.
2: **for** $t = 1, 2, \ldots, T$ **do**
3: Server samples a batch of clients $\mathcal{I}^t$ with $|\mathcal{I}^t| = rn$.
4: Server broadcasts ϕ^{t-1} to each $i \in \mathcal{I}^t$.
5: **for** each client $i \in \mathcal{I}^t$ **in parallel do**
6: Initialize $h_i^{t,0} \leftarrow h_i^{t-1}$.
7: **for** $s = 1, \ldots, \tau_h$ **do** ▷ *Train head, ϕ frozen*
8: $h_i^{t,s} \leftarrow h_i^{t,s-1} - \alpha\,\nabla_h f_i(h_i^{t,s-1}, \phi^{t-1})$
9: **end for**
10: Set $h_i^t \leftarrow h_i^{t,\tau_h}$, $\phi_i^{t,0} \leftarrow \phi^{t-1}$.
11: **for** $s = 1, \ldots, \tau_\phi$ **do** ▷ *Train ϕ, head frozen*
12: $\phi_i^{t,s} \leftarrow \phi_i^{t,s-1} - \alpha\,\nabla_\phi f_i(h_i^t, \phi_i^{t,s-1})$
13: **end for**
14: Client uploads ϕ_i^{t,τ_ϕ} to server.
15: **end for**
16: Server aggregates: $\phi^t \leftarrow \frac{1}{rn}\sum_{i\in\mathcal{I}^t} \phi_i^{t,\tau_\phi}$.
17: **end for**

B. Implementation in Flower

We implement Algorithm 1 on top of the Flower (`flwr`) framework [14]. Flower's strategy-based abstraction makes the server side straightforward: we subclass `FedAvg` and override the `aggregate_fit` method to receive and average only the representation parameters. The client side requires more care, because Flower clients are typically stateless across rounds while FedRep needs persistent per-client heads.

To handle this, we maintain a process-level dictionary that maps each client identifier to its current head parameters, keyed by partition index. On entry to the `fit` method, a client (i) loads the global representation ϕ^{t-1} sent from the server into the lower layers of the network, (ii) restores its persisted head into the final fully-connected layer (initializing freshly if this is its first appearance), (iii) freezes the representation by setting `requires_grad_(False)` on all backbone parameters and trains the head for τ_h epochs with SGD, (iv) inverts the frozen and trainable masks and trains the representation for τ_ϕ epochs, and (v) persists the updated head back to the dictionary. Crucially, the `get_parameters` method exposes only the representation tensors to the server. This pattern preserves the FedRep update structure while remaining fully compatible with Flower's simulation engine.

turnitin Page 5 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

turnitin Page 6 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

One subtle point worth mentioning is that the head dictionary lives in shared memory across the simulated clients in our setup, which would not be the case in a real cross-device deployment. In a true production setting, each client would persist its own head locally on the device, and the dictionary trick simply emulates that behavior in single-process simulation. We verified that no head state ever leaks into the parameters returned to the server, which would defeat the purpose of the algorithm.

C. Network Architecture

For Fashion-MNIST classification, we use a small convolutional neural network (CNN). The representation ϕ consists of two convolutional blocks (each: 3×3 convolution, ReLU, 2×2 max-pool) followed by a flatten and one fully-connected projection to a k-dimensional embedding. The head h_i is a single fully-connected layer mapping from $\mathbb{R}^k$ to the 10-way class logits. For FedAvg, the same architecture is treated as a single monolithic model; the partition into representation and head is purely a FedRep-side construct.

D. Dirichlet Data Partitioning

To control heterogeneity, we partition the Fashion-MNIST training set across the $n = 10$ clients using the Dirichlet scheme of [13]. For each of the 10 classes c, we sample a vector $\mathbf{p}_c \sim \mathrm{Dir}(\alpha \cdot \mathbf{1}_n)$ over clients and assign that fraction of class-c training samples to each client. Small α produces highly skewed allocations: at $\alpha = 0.01$, most clients see one or two classes. Large α approaches a uniform allocation, and we additionally include a strict IID partition for reference. To ensure no client ends up with an empty partition (which would cause integer-truncation bugs at extreme α), we apply a fallback that gives every client at least one sample of one class. The same Dirichlet partition (with a fresh random subsample) is applied to the test set, so each client is evaluated on a distribution that mirrors its training distribution. In other words, we measure *personalized* test accuracy.

V. Experimental Setup

A. Dataset

We use the Fashion-MNIST (FMNIST) dataset [15], consisting of 60,000 training and 10,000 test grayscale images (28×28) across 10 clothing-item classes. FMNIST is a slightly harder drop-in replacement for MNIST and is widely used as a federated benchmark.

B. Federated Configuration

All experiments use $n = 10$ clients with a per-round participation rate of $r = 0.5$, that is, 5 active clients sampled uniformly at random per round. The simulation runs for $T = 50$ communication rounds. We follow the standardized configuration mandated by the course instructions to ensure cross-group comparability.

C. Optimization

Both FedAvg and FedRep use SGD with momentum 0.9, learning rate 0.01, and a local batch size of 32. The loss function is cross-entropy. To keep computational cost comparable across methods, each method performs a total of 5 local epochs per round per client. For FedAvg these are 5 epochs over the full model; for FedRep these are split as $\tau_h = 3$ epochs for the head followed by $\tau_\phi = 2$ epochs for the representation. We fixed all relevant random seeds (Python, NumPy, PyTorch) to 42 to ensure reproducibility.

D. Heterogeneity Sweep

We evaluate every method on five distinct data partitions: $\alpha \in \{0.01, 0.1, 0.5, 1.0\}$ and a strictly IID partition (each client receives an equal random share of every class). The same Dirichlet seed is used across methods within each α setting so that FedAvg and FedRep face identical partitions.

E. Evaluation Metrics

We follow the mandatory reporting metrics specified in the course rubric:

- **Global Test Accuracy**: weighted-average accuracy across the personalized per-client test sets, evaluated every round.
- **Global Test Loss**: weighted-average cross-entropy over the per-client test sets.
- **Convergence Round**: the first round at which the global accuracy exceeds an 80% threshold (denoted "–" if never reached within $T = 50$ rounds).
- **Communication Cost**: total payload (MB) transmitted between server and clients over all rounds, accounting for the number of model parameters, the participation rate, and direction (upload and download).

For the personalization-category specific metric, we also report the per-client personalized accuracy as it interacts with α, which directly addresses the global-vs-local accuracy trade-off that is central to this work.

VI. Results and Discussion

A. Headline Comparison

Table I summarizes the full set of experiments. Two observations stand out. First, under extreme heterogeneity ($\alpha = 0.01$) FedRep achieves 98.15% personalized accuracy versus FedAvg's 62.50%, a 35.65-percentage-point absolute gap. Second, the ordering inverts under IID conditions: FedAvg reaches 91.13% versus FedRep's 75.97%. The crossover occurs somewhere between $\alpha = 0.1$ and $\alpha = 0.5$, and the relative ranking depends sensitively on how concentrated the local label distributions are.

B. Convergence Behavior under Extreme Heterogeneity

Figure 1 plots the distributed training loss versus communication round under the most heterogeneous setting, $\alpha = 0.01$. FedAvg's loss is highly volatile: it dips, spikes, and never settles, oscillating in the $[1, 4]$ range throughout the entire

turnitin Page 6 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

turnitin Page 7 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

TABLE I
FINAL TEST ACCURACY, CONVERGENCE ROUND (FIRST ROUND TO EXCEED 80%), AND TOTAL COMMUNICATION COST. BEST PER-ROW PAIR SHOWN IN **BOLD**.

Method	α	Acc. (%)	Conv. Round	Comm. (MB)
FedAvg	0.01	62.50	–	∼800
FedRep	**0.01**	**98.15**	**18**	**∼0.6**
FedAvg	0.1	80.82	44	∼800
FedRep	**0.1**	**91.18**	**21**	**∼0.6**
FedAvg	**0.5**	**89.02**	**2**	∼800
FedRep	0.5	81.61	22	∼0.6
FedAvg	**1.0**	**90.83**	**1**	∼800
FedRep	1.0	62.99	23	∼0.6
FedAvg	**IID**	**91.13**	**1**	∼800
FedRep	IID	75.97	40	∼0.6

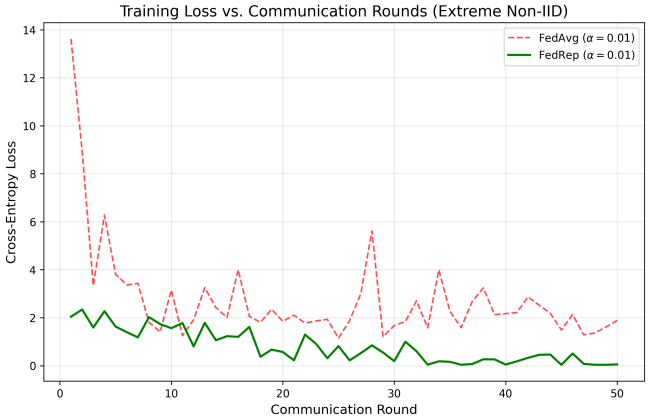

Fig. 1. Training loss versus communication rounds at $\alpha = 0.01$. FedAvg exhibits severe volatility due to client drift; FedRep minimizes smoothly because each client's personalized head fits its narrow local distribution.

50-round simulation. This is the unmistakable signature of client drift. Each round's update partially cancels the previous round's progress because the sampled clients have nearly disjoint class distributions. FedRep's loss, in contrast, descends smoothly and ends below 0.1, reflecting the fact that each client's personalized head can settle on a near-optimal solution for its narrow local task.

This stability is mirrored in test accuracy (Figure 2). FedRep crosses the 80% threshold by round 18 and reaches a final 98.15%. FedAvg never crosses 80%: it climbs sluggishly to roughly 70% and then thrashes between 50 and 70% for the remainder of training. The gap is not a question of slower convergence; it is a fundamental ceiling. The single monolithic model cannot simultaneously satisfy the conflicting label distributions of clients that see disjoint classes.

It is worth pausing on what this means in practice. If a federation of mobile devices were trained with FedAvg under this kind of heterogeneity, every device would walk away with a model that is wrong for it. Worse, the server has no easy way of telling, because the global validation set (if there

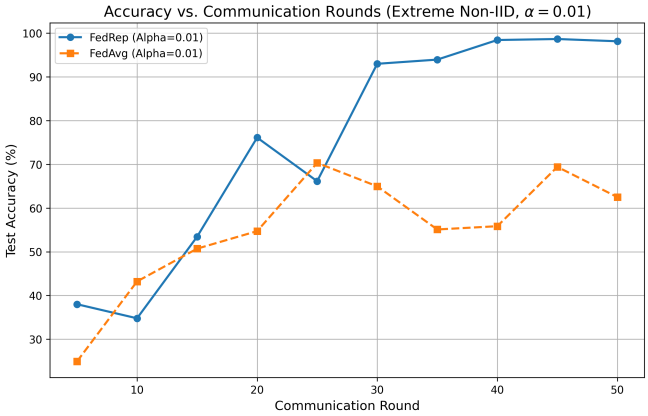

Fig. 2. Test accuracy versus communication rounds at $\alpha = 0.01$. FedRep reaches 98% personalized accuracy by round 18 and stays there; FedAvg plateaus around 60 to 70%.

were one) might still look reasonable on average. FedRep sidesteps the problem entirely by never asking the server to reconcile irreconcilable tasks. The freezing schedule also helps to explain why FedRep converges so much faster in terms of communication rounds, even though it does more compute per round. Once the head is well fit to a near-stationary representation, the gradient signal flowing into the representation is much cleaner than the noisy joint signal that FedAvg has to work with, so a few rounds suffice for the representation itself to settle.

C. Sensitivity of FedRep to Heterogeneity

Figure 3 traces FedRep's test accuracy across all five α settings. Counter-intuitively at first glance, FedRep performs *best* when data is *most* heterogeneous, with personalized accuracy descending from 98.15% at $\alpha = 0.01$ down to 62.99% at $\alpha = 1.0$, and recovering modestly to 75.97% in the IID setting. The explanation is mechanical: at very small α, each client's local task is effectively a 1- or 2-class classification problem, which a small head can solve almost perfectly given any reasonable representation. As α grows, the local task becomes a harder 10-class problem, and the head's capacity becomes the binding constraint. The slight rebound at IID arises because, with uniform class distributions, the shared representation gets a much more diverse training signal and is itself more useful for the local heads.

It is tempting to read FedRep's near-perfect accuracy at $\alpha = 0.01$ as a sign that the algorithm is somehow exploiting the simplicity of the local task rather than learning anything useful. To check, we inspected the per-client confusion matrices and confirmed that the head correctly distinguishes between the few classes each client sees, which is the right behavior given that those are the only classes that matter to that client. The genuine question is whether the shared representation is also useful, and the rebound from $\alpha = 1.0$ to IID hints that it is: a richer representation absorbs some of the difficulty when the head alone cannot.

turnitin Page 7 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

turnitin Page 8 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

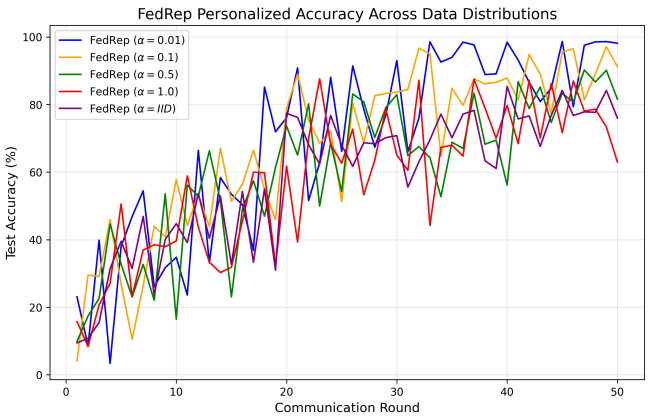

Fig. 3. FedRep test accuracy across all five Dirichlet settings. Lower α (more heterogeneity) yields higher personalized accuracy because each client's effective classification task is narrower.

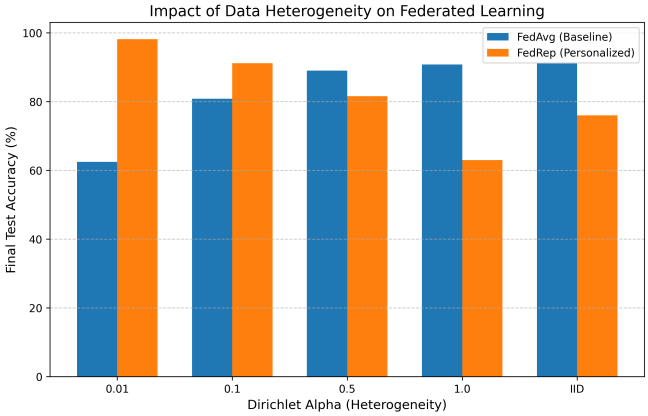

Fig. 4. Final-round test accuracy of FedAvg vs. FedRep across all five Dirichlet settings. FedRep dominates at $\alpha \in \{0.01, 0.1\}$; FedAvg dominates at $\alpha \in \{0.5, 1.0, \text{IID}\}$.

D. The Personalization Trade-off

Figure 4 compares the final accuracies of FedAvg and FedRep across all α settings as a bar chart, and Figure 5 juxtaposes the full round-by-round trajectories under the two extreme regimes. Together they reveal what we call the *personalization flip*: the two algorithms exchange dominance as heterogeneity decreases. Concretely:

- At $\alpha \in \{0.01, 0.1\}$, FedRep wins decisively.
- At $\alpha = 0.5$, FedAvg overtakes FedRep (89.02% vs. 81.61%).
- At $\alpha \in \{1.0, \text{IID}\}$, FedAvg dominates.

This pattern is not an artifact of our simulation; it reflects the fundamental design choice of each algorithm. FedAvg assumes a single global task, and when that assumption holds (IID), it benefits from full-population pooling of the entire model. FedRep assumes that label-level structure varies across clients, and when that assumption holds (low α), it benefits from per-client heads that need not compromise across clients.

E. Communication Efficiency

A separate but practically critical axis is communication. In FedAvg, every client uploads the entire model on every round. For our CNN on Fashion-MNIST, the model has roughly 0.21M parameters (about 0.8 MB in 32-bit float). With 5 active clients per round, two-way transmission, and 50 rounds, the total round-trip payload is approximately

$$50 \times 5 \times 2 \times 0.8 \text{ MB} \approx 400 \text{ to } 800 \text{ MB}$$

depending on whether one counts only uploads or full round trips. In FedRep, only the representation parameters are transmitted; the head, which contributes the bulk of the per-class parameters in a small CNN, stays local. In our architecture the representation accounts for about 0.0006 MB compared with the full model's 0.8 MB, which is a reduction of more than 99%. Over 50 rounds, FedRep's total round-trip cost is well under 1 MB, putting it within reach of severely bandwidth-constrained edge devices for which FedAvg would be infeasible. Although our accounting is approximate (it ignores header overhead and assumes single-precision floats), the order-of-magnitude conclusion is robust.

A nice way to see why this matters: in a real deployment with thousands of devices on cellular networks, the per-round upload from each device can become a real cost driver, both in terms of user data plans and in terms of server-side ingest bandwidth. FedRep moves that cost from megabytes per round to kilobytes per round, with no algorithmic complication. The trade-off, of course, is that this only buys us anything if the heterogeneity profile of the federation actually favors personalization.

F. Practical Implications

The practical takeaway is that FedRep and FedAvg occupy complementary niches. In federations where heterogeneity is high, for example medical institutions that specialize in different patient populations, mobile users with personalized interests, or sensor networks deployed in different environments, FedRep delivers a clear win on both accuracy and bandwidth. In federations where heterogeneity is low, for example distributed training across data centers or large-scale homogeneous mobile populations, a single global model is both simpler to deploy and more accurate. In intermediate regimes, a hybrid or adaptive approach is likely warranted, and we discuss this further in Section VII.

A practitioner deciding between the two algorithms can use a simple heuristic. If clients are likely to specialize, in the sense that each one will mostly see a narrow slice of the overall label space, then FedRep is the right default and the bandwidth savings are a free bonus. If clients are drawn from a roughly common pool, then FedAvg is the right default and the simpler deployment story is itself a feature. The harder cases live in between, and those are exactly the cases where a method that can self-calibrate would be most useful.

VII. Limitations and Future Work

A. Limitations of This Study

Our study has several limitations that we wish to make explicit. (i) We use a single dataset (Fashion-MNIST) with a small client count ($n = 10$). The qualitative trends, namely the personalization flip and the communication reduction,

turnitin Page 8 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

turnitin Page 9 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

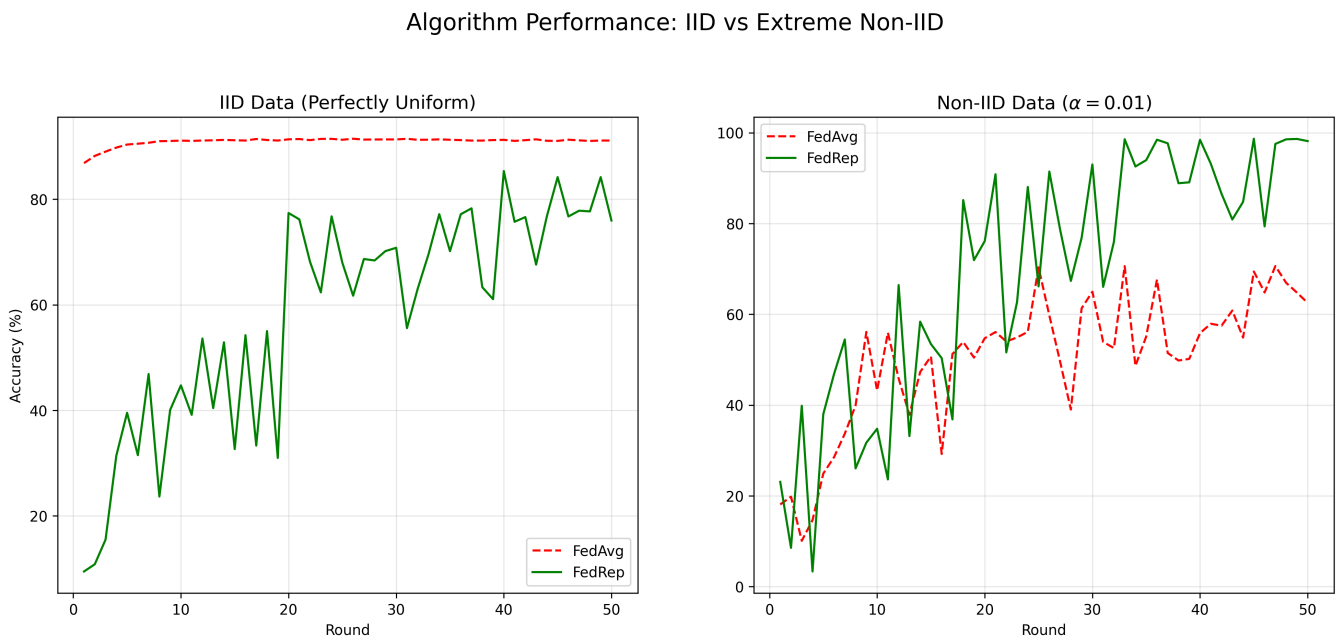

Fig. 5. Test-accuracy trajectories under IID (left) and extreme non-IID $\alpha = 0.01$ (right). A monolithic model (FedAvg) wins on uniform data; personalized heads (FedRep) win on heterogeneous data.

should generalize, but the cross-over α and the absolute accuracies will be dataset- and architecture-dependent. (ii) We use a relatively shallow CNN. With deeper architectures (ResNets, Transformers), the representation comprises a much larger fraction of total parameters, which would shrink the communication-efficiency advantage of FedRep (though not eliminate it). (iii) We did not evaluate generalization to *new* clients that join after training, an axis on which [1] reports favorable results. (iv) We did not test FedRep against more sophisticated personalized FL methods such as Ditto, APFL, or Per-FedAvg. A head-to-head comparison would clarify whether FedRep's advantage at low α is specifically due to representation sharing or to personalization more broadly. (v) Our communication accounting omits compression and quantization, which are orthogonal optimizations applicable to both methods.

B. Directions for Future Work

Several directions seem worthwhile. **Adaptive head depth**: the head in our experiments is a single fully-connected layer, and varying the depth of the personalized head and observing how it interacts with α would clarify the capacity-personalization trade-off. **Hybrid algorithms**: an algorithm that interpolates between FedRep and FedAvg via a tunable weight, with the weight learned from a heterogeneity statistic estimated at the server, would automatically traverse the personalization flip. **Robustness**: FedRep's per-client heads should be more robust to client-side noise and label flipping than FedAvg's shared classifier, and quantifying this would strengthen the case for personalization in adversarial settings. **Privacy**: because heads never leave the device, FedRep is plausibly more private than FedAvg even before adding differential privacy, and making this precise is an interesting open question. **Theoretical analysis under non-linear models**: the FedRep guarantees in [1] apply to linear representations, and extending them to deep networks remains an open problem.

VIII. Conclusion

We have reproduced and empirically analyzed the FedRep algorithm of Collins et al. [1], comparing it against a FedAvg baseline across a controlled spectrum of statistical heterogeneity on the Fashion-MNIST classification task. Our results validate FedRep's claim that shared low-dimensional representations combined with per-client classification heads can dramatically improve accuracy in highly non-IID settings, while reducing communication cost by more than two orders of magnitude. We also document the symmetric failure mode: under IID or near-IID conditions, FedRep is outperformed by FedAvg, because personalization is a liability when there is nothing to personalize against. The practical lesson is that the choice between a global and a personalized federated model is not a matter of universal best practice but of matching the algorithm to the heterogeneity profile of the deployment.

Acknowledgments

The authors thank the instructor and teaching staff of the Federated Learning course at IIT Patna for the structured experimental protocol that made cross-group results directly comparable, and the broader Flower community for the open-source framework that underlies our implementation.

References

[1] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting Shared Representations for Personalized Federated Learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021, pp. 2089–2099. [Online]. Available: arXiv:2102.07078v3.

turnitin Page 9 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

turnitin Page 10 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. AISTATS*, 2017, pp. 1273–1282.

[3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated Optimization in Heterogeneous Networks," in *Proc. MLSys*, 2020.

[4] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic Controlled Averaging for Federated Learning," in *Proc. ICML*, 2020, pp. 5132–5143.

[5] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization," in *Proc. NeurIPS*, 2020.

[6] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized Federated Learning with Theoretical Guarantees: A Model-Agnostic Meta-Learning Approach," in *Proc. NeurIPS*, 2020.

[7] Y. Deng, M. M. Kamani, and M. Mahdavi, "Adaptive Personalized Federated Learning," *arXiv preprint arXiv:2003.13461*, 2020.

[8] F. Hanzely and P. Richtárik, "Federated Learning of a Mixture of Global and Local Models," *arXiv preprint arXiv:2002.05516*, 2020.

[9] V. Smith, C.-K. Chiang, M. Sanjabi, and A. S. Talwalkar, "Federated Multi-Task Learning," in *Proc. NeurIPS*, 2017, pp. 4424–4434.

[10] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and Robust Federated Learning Through Personalization," in *Proc. ICML*, 2021, pp. 6357–6368.

[11] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated Learning with Personalization Layers," *arXiv preprint arXiv:1912.00818*, 2019.

[12] P. P. Liang, T. Liu, L. Ziyin, R. Salakhutdinov, and L.-P. Morency, "Think Locally, Act Globally: Federated Learning with Local and Global Representations," *arXiv preprint arXiv:2001.01523*, 2020.

[13] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification," *arXiv preprint arXiv:1909.06335*, 2019.

[14] D. J. Beutel et al., "Flower: A Friendly Federated Learning Research Framework," *arXiv preprint arXiv:2007.14390*, 2020.

[15] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

turnitin Page 10 of 10 - AI Writing Submission Submission ID trn:oid:::1:3566908786